

Note: Some images in this report may appear small due to layout constraints. Please zoom in when viewing. All screenshots are clean and clearly show the forensic evidence acquired for analysis.

Table 1: Tools utilized for Forensic Investigation

Tool	Purpose
sha256sum	Verified the integrity of the memory image by comparing its hash against the provided reference.
Volatility 2	Core memory analysis tool used for process enumeration (<code>pslist</code> , <code>psscan</code>), code injection detection (<code>malfind</code>), DLL inspection (<code>dlllist</code>), process dumping (<code>dumpfiles</code>), and network connection analysis (<code>connections</code>).
strings + grep	Extracted readable strings from dumped memory and filtered for artefacts such as IPs, emails, URLs, and commands.
less, cat, sort	Used to review, organize, and deduplicate extracted artefacts for analysis.

1 Task 1: Preparation and Verification of Integrity

To verify the integrity of `memdump.raw` by comparing its hash against hash value in `memdumphash.txt`, the following was done:

Navigate to working directory and list investigation files

```
cd ~/Desktop/25757637_Prac4
ls -lh
```

1
2

View `memdumphash.txt` file contents

```
memdumphash.txt
```

```
sha256sum hash command
```

```
sha256sum memdump.raw
```



Figure 1: Screenshot of executed commands and outputs

1.1 Findings

The computed hash of memdump.raw matched the expected value in mem-dumphash.txt, confirming that the image has not been altered.

2 Task 2: Process Analysis

2.1 Running processes

```
python2 vol.py -f ~/Desktop/25757637_Prac4/memdump.raw
--profile=WinXPSP3x86 pslist
```

1

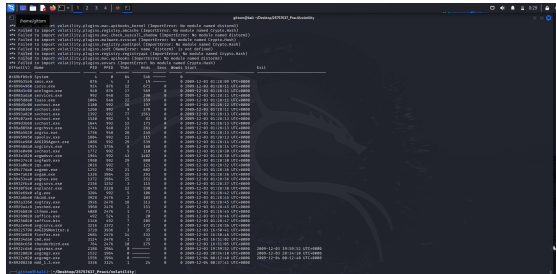


Figure 2: Screenshot- output of running processes

2.2 Legitimate system processes

- System, smss.exe, csrss.exe, winlogon.exe, services.exe, lsass.exe are all expected for Windows XP.
- Multiple svchost.exe instances are normal for service hosting.

2.3 AVG Antivirus processes

- avgchsvx.exe, avgrsx.exe, avgcsrvx.exe, avgwdsvc.exe, avgfws9.exe, avg-tray.exe, AVGIDSAgent.exe, AVGIDSMonitor.e, avgemc.exe, avgam.exe, avgnsx.exe, avgsrmax.exe, avgcmgr.exe are consistent with the AVG suite.

2.4 Suspicious and noteworthy entries

- Multiple avgcsrvx.exe instances with different PPIDs may be legitimate, but worth checking for duplication or injection.
- Processes with 0 threads and no handles:
 - avgsrmax.exe, avgcmgr.exe (x2) processes exited immediately, which is possibly suspicious.
- mdd.1.3.exe is not a standard Windows or AVG process. It could be forensic software or something injected.
- explorer.exe has PPID 2228, but the parent is not listed, possibly terminated or hidden.

2.5 Hidden or unlinked processes

```
python2 vol.py -f ~/Desktop/25757637_Prac4/memdump.raw
--profile=WinXPSP3x86 psscan
```

1

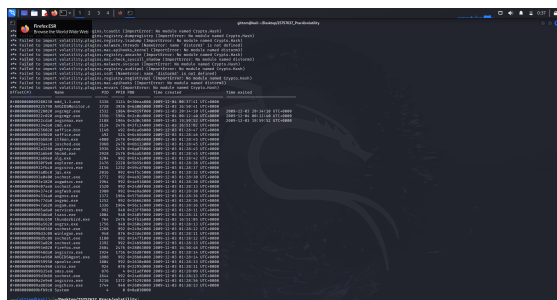


Figure 3: Screenshot- output of hidden or unliked processes

2.6 Suspicious indicators

- Duplicate executables: avgcmgr.exe appears twice with different PIDs and identical PPID.

- Immediate exit: `avgcmgr.exe` and `avgsrcmax.exe` exited at creation, meaning they could be injected or failed startup.
- Unusual names: `AVGIDSMonitor.e` is truncated, possibly obfuscated or corrupted.
- Missing from `pslist`: These processes were not tracked by the OS at the time of capture.

2.7 Identified suspicious processes

- `avgcmgr.exe` (1532, 1556): Duplicate instances, exited instantly, not in `pslist`
- `avgsrcmax.exe` (2188): Zero threads, immediate exit
- `AVGIDSMonitor.e` (3728): Truncated name, possible obfuscation
- `explorer.exe` (2476): Parent PID missing, breaks hierarchy
- `avgcsrpx.exe`: Multiple instances with different PPIDs which are worth monitoring
- `mdd_1.3.exe`: Non-standard binary that validate against forensic tools

These anomalies suggest potential stealth behavior, process injection, or tampering. Further investigation is recommended to determine legitimacy.

3 Task 3: Code Injection and DLL Analysis

3.1 windows.malfind -Check injected code on processes

```
python2 vol.py -f ~/Desktop/25757637_Prac4/memdump.raw  
--profile=WinXPSP3x86 malfind
```

1



Figure 4: Screenshot- csrss.exe process

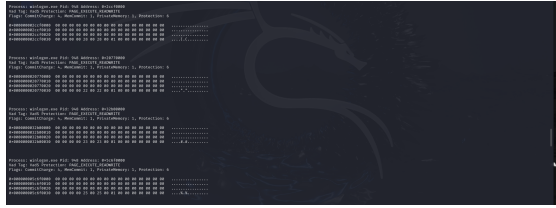


Figure 5: Screenshot- winlogon.exe processes

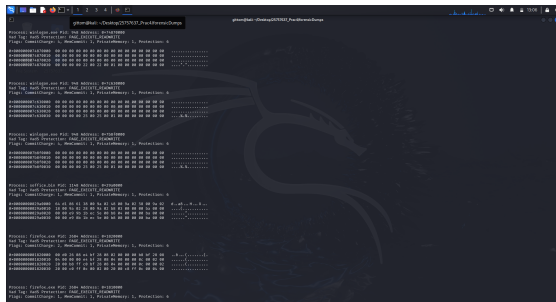


Figure 6: Screenshot- soffice.bin processes

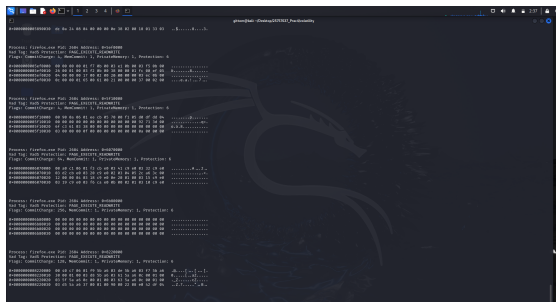


Figure 7: Screenshot- firefox.exe processes

The `windows.malfind` plugin identified multiple processes with suspicious memory traits and injected regions:

- `csrss.exe` (PID 924): A system-critical process containing an executable memory page not backed by a file — highly suspicious.
- `winlogon.exe` (PID 948): Over eight injected regions suggest stealth injection or malware targeting session management.
- `soffice.bin` (PID 1148): Structured binary data in an executable region may indicate embedded payloads or shellcode.

- **firefox.exe** (PID 2684): Numerous regions with executable permissions, some zeroed out and others structured — potential plugin abuse, injection, or persistence mechanisms.

3.2 Dumping suspicious processes

3.2.1 PID =948 (winlogon.exe)

```
python2 vol.py -f ~/Desktop/25757637_Prac4/memdump.raw  
--profile=WinXPSP3x86 dumpfiles --pid=948  
--dump-dir=~/Desktop/25757637_Prac4/forensicDumps
```

1

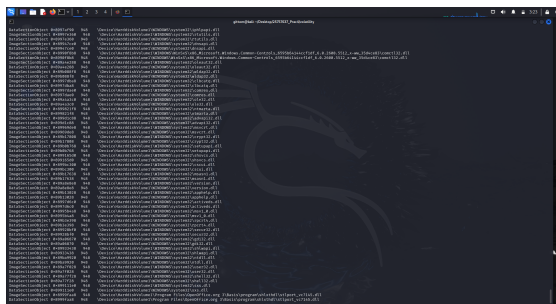


Figure 8: Screenshot- winlogon.exe process dump

Over 90 memory regions were dumped from **winlogon.exe**, including:

- The executable itself: **winlogon.exe**
- Multiple DLLs: **netapi32.dll**, **ws2_32.dll**, **WgaLogon.dll**, **avgrssth.dll**, **mpr.dll**, **shell32.dll**, **ntdll.dll**, etc.
- NLS files: **unicode.nls**, **locale.nls**, **sorttbls.nls**, etc.
- OpenOffice-related DLL: **stlport_vc7145.dll** - unexpected in a system process
- The presence of **avgrssth.dll** and **WgaLogon.dll** may indicate third-party or injected components.
- The inclusion of OpenOffice DLLs (**stlport_vc7145.dll**) in **winlogon.exe** is highly suspicious. This process should not load user-level application libraries.
- These anomalies support the findings from **malfind** - multiple executable regions with no file backing.

3.2.2 PID =924 (csrss.exe)

```
python2 vol.py -f ~/Desktop/25757637\_Prac4/memdump.raw  
--profile=WinXPSP3x86 dumpfiles --pid=924  
--dump-dir=~ /Desktop/25757637\_Prac4/forensicDumps}
```

1

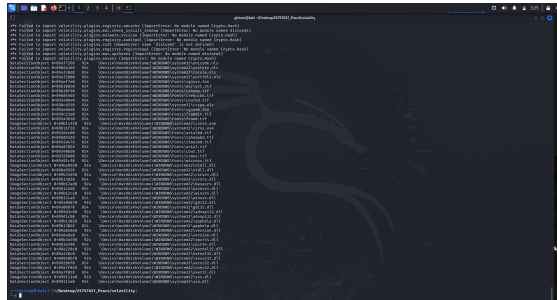


Figure 9: Screenshot- ccsrss.exe process dump

The memory dump of `ccrss.exe` included the following components:

- Executable image: `ccrss.exe`
- DLLs and support libraries: `ntdll.dll`, `ccsrv.dll`, `basesrv.dll`, `winsrv.dll`, `advapi32.dll`, `rpcrt4.dll`, `kernel32.dll`, `user32.dll`, etc.
- Font files: `arial.ttf`, `cour.ttf`, `micross.ttf`, `marlett.ttf`, etc.
- NLS files: `unicode.nls`, `locale.nls`, `ctype.nls`, etc.
- The dump includes all expected system components for `ccrss.exe`.
- No third-party DLLs or unexpected mappings were found — unlike `winlogon.exe`, which had OpenOffice and AVG components.
- However, since `malfind` flagged `ccrss.exe` for executable memory not backed by a file, the dumped image remains a priority for hashing and inspection.

3.2.3 PID =1148 (sooffice.bin)

```
python2 vol.py -f ~/Desktop/25757637\_Prac4/memdump.raw  
--profile=WinXPSP3x86 dumpfiles --pid=1148  
--dump-dir=~ /Desktop/25757637\_Prac4/forensicDumps}
```

1



Figure 10: Screenshot- soffice.bin process dump

The memory dump of `soffice.bin` revealed the following components:

- Executable image: `soffice.bin`
- OpenOffice DLLs: `sofficeapp.dll`, `libxml2.dll`, `oooimprovementmi.dll`, `fwemi.dll`, `fwimi.dll`, `comphelp4MSC.dll`, `cppu3.dll`, `bootstrap.uno.dll`, etc.
- Resource files: `.res`, `.rdb`, `.uno.dll`
- System DLLs: `ntdll.dll`, `kernel32.dll`, `user32.dll`, `gdi32.dll`, `advapi32.dll`, `rpcrt4.dll`, `shell32.dll`, etc.
- Unexpected mappings: `unsafe_uno_uno.dll`, `stlport_vc7145.dll`, `jvmfwk3.dll`, `purpenvhelper3MSC.dll`
- The dump confirms that `soffice.bin` loaded a large number of OpenOffice components, including several DLLs not typically associated with benign document editing.
- The presence of `unsafe_uno_uno.dll` and `stlport_vc7145.dll` aligns with the suspicious memory flagged by `malfind`.
- These files are high-priority candidates for SHA256 hashing and further inspection.

3.2.4 PID =2684 (firefox.exe)

```
python2 vol.py -f ~/Desktop/25757637_Prac4/memdump.raw
--profile=WinXPSP3x86 dumpfiles --pid=2684
--dump-dir=~/Desktop/25757637_Prac4/forensicDumps
```

1



Figure 11: Screenshot- firefox.exe process

The memory dump of `firefox.exe` revealed the following components:

- Executable image: `firefox.exe`
- Firefox components: `xul.dll`, `sqlite3.dll`, `js3250.dll`, `nss3.dll`, `smime3.dll`, `ssl3.dll`, `mozcr19.dll`, etc.
- User profile artifacts:
 - Databases: `places.sqlite`, `cookies.sqlite`, `downloads.sqlite`, `webappsstore.sqlite`, `signons.sqlite`, `cert8.db`, `key3.db`
 - Cache files: `_CACHE_001_`, `_CACHE_MAP_`, etc.
- System DLLs: `ntdll.dll`, `kernel32.dll`, `user32.dll`, `shell32.dll`, `advapi32.dll`, `rpcrt4.dll`, `gdi32.dll`, etc.
- The dump includes both application binaries and user-level data, such as browsing history, saved passwords, and cookies.
- These artifacts are highly valuable for forensic analysis and should be prioritized for hashing and inspection.
- The presence of multiple `.sqlite` and `.db` files suggest potential privacy or credential exposure.

3.3 SHA256 hashes of dumped executables

Detection of embedded executable files via DOS mode signature

```
grep -l "This program cannot be run in DOS mode" .img .dat
```

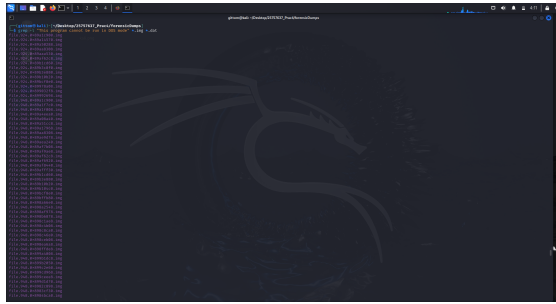


Figure 12: Output of indicator of a Windows executable (PE file) embedded in memory or disk artefacts

SHA256 hashing of embedded PE files

```
sha256sum $(grep -l "This program cannot be run in DOS mode"
.img .dat) > hashes.txt
```

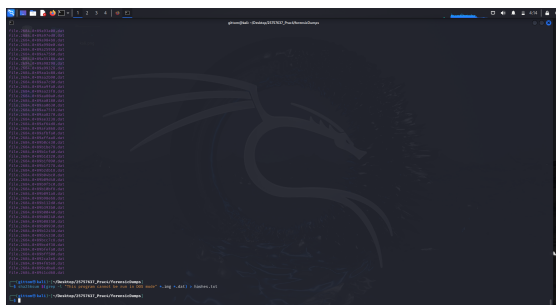
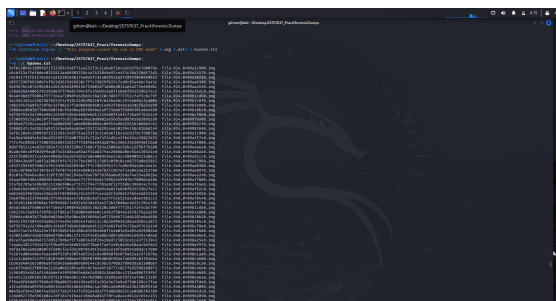


Figure 13: Screenshot- Output

Hashes view command

```
cat hashes.txt
```



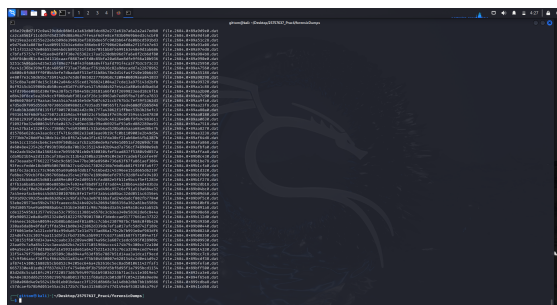


Figure 14: Screenshot- hashed values

3.4 Review loaded DLLs for anomalies

```
python2 vol.py -f ~/Desktop/25757637_Prac4/memdump.raw
--profile=WinXPSP3x86 dlllist >
~/Desktop/25757637_Prac4/dlllistOutput.txt
```

```
less ~/Desktop/25757637_Prac4/dlllistOutput.txt
```

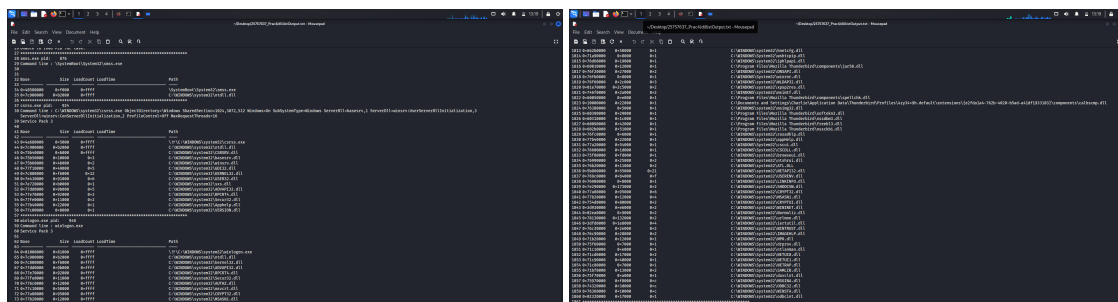


Figure 15: Screenshot - DLLs output

The `dlllist` plugin was used to enumerate loaded DLLs across active processes. The review focused on missing paths, unusual names, non-standard directories, and injection into critical system processes.

3.5 Most DLLs were loaded from standard Windows directories or trusted software:

- Examples: AVG Antivirus, Firefox, Thunderbird, OpenOffice, Java, Intel drivers, forensic tools
- All DLLs had valid paths and recognizable names
- No signs of injection, hollowing, or reflective loading

3.6 Notable anomalies

- `avgrssth.dll` in `winlogon.exe` (PID 948) — unusual presence in a core authentication process; likely benign but worth documenting
- Inaccessible AVG processes (PIDs 1532, 1556, 2188) — unable to read PEB; no DLLs enumerated

DLL loading appears legitimate overall. The only anomaly involves `avgrssth.dll` in `winlogon.exe`, which warrants attention due to its elevated context.

4 Task 4: Network and Artefact Discovery

4.1 Sockets

```
python2 vol.py -f ~/Desktop/25757637_Prac4/memdump.raw
--profile=WinXPSP3x86 sockets
```

1

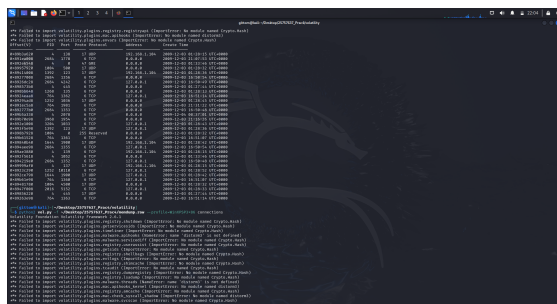


Figure 16: Screenshot- sockest found

4.1.1 Socket activity observations

- PID 2684 () shows extensive socket activity, including multiple loopback connections and listening ports. This is consistent with browser behavior but should be correlated with strings analysis to check for suspicious URLs or IPs.
- PID 764 () has several loopback sockets — expected for a mail client, but worth checking for artefacts in its dump.
- PID 3968 () having a TCP socket is unusual for a Java update scheduler. This warrants further investigation.

- PID 1004 with UDP port 4500 is notable — commonly used in VPN or IPsec tunneling. If this system wasn't expected to run such services, it may be suspicious.
- System process (PID 4) has NetBIOS and SMB ports open — expected on XP but sensitive in forensic contexts.

4.1.2 Connections

```
python2 vol.py -f ~/Desktop/25757637_Prac4/memdump.raw
--profile=WinXPSP3x86 connections
```

1

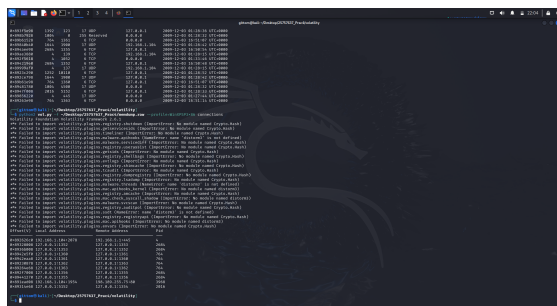


Figure 17: Screenshot- connections found

4.2 Network connections observations

The plugin revealed several active network connections:

- (PID 3968) connected to an external IP over HTTP (port 80). This is suspicious, as Java update schedulers typically do not initiate outbound connections. This may indicate misuse or masquerading.
- (PID 4) connected over SMB (port 445). While expected in XP environments, this connection should be documented due to its potential for remote access.
- Loopback connections for (PID 2684) and (PID 764) are consistent with normal browser and mail client behavior.

4.3 Discovering Artefacts

Regex artefacts scan command

```
strings .img | grep -Ei '([0-9]{1,3}){3}[0-9]{1,3}|[A-Za-z0-9._-
%+-]+@[A-Za-z0-9.-]+[A-Za-z]{2},|http[s]?://[^\s|]+\www[^\s|]+|cmdexe|
powershell|ftp|netstat|whoami|tasklist|ping|curl|wget' >
artefacts.txt
```

The scan command extracts readable strings from dumped process memory and filters for forensic artefacts such as IPs, emails, URLs, and shell commands. It uses regular expressions to isolate attacker-relevant indicators and saves them to the artefacts.txt text file for structured analysis.

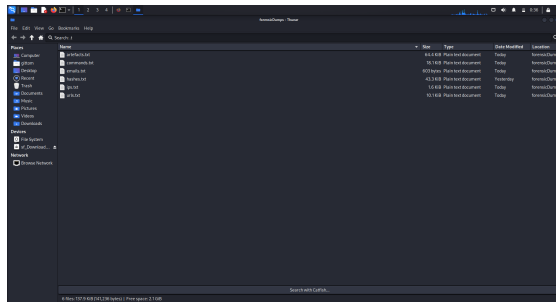


Figure 18: Screenshot- output

4.3.1 IP Artefacts

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' artefacts.txt | sort -u >
ips.txt
```

1

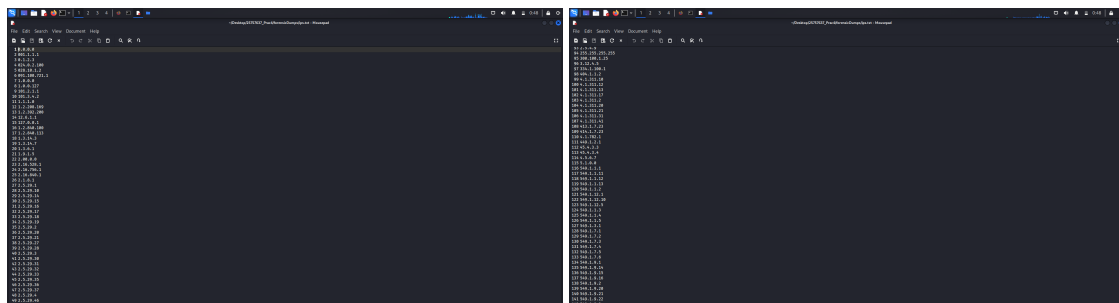


Figure 19: Screenshots- output

These are syntactically valid IPv4 addresses:

- 0.0.0.0 - Often used as a placeholder or wildcard address
- 127.0.0.1 - Loopback (local host)

- 255.255.255.255 - Broadcast address
- 45.4.3.3, 45.4.3.4 - Potential external IPs and are worth investigating
- 101.2.1.1, 101.3.4.2, 1.2.208.169 - Possibly routable and can be checked against known attacker IPs

The following are either malformed or not real IPs:

- 091.100.721.1, 300.100.1.25, 404.1.1.2, 334.1.100.1 - Invalid octets above 255
- 1.2.840.100, 2.5.29.14, 4.1.311.10, etc. - These are OID (Object Identifiers), not IPs. They appear in certificate metadata and ASN.1 structures, suggesting the presence of cryptographic or authentication structures within memory.

4.3.2 Email addresses Artefacts

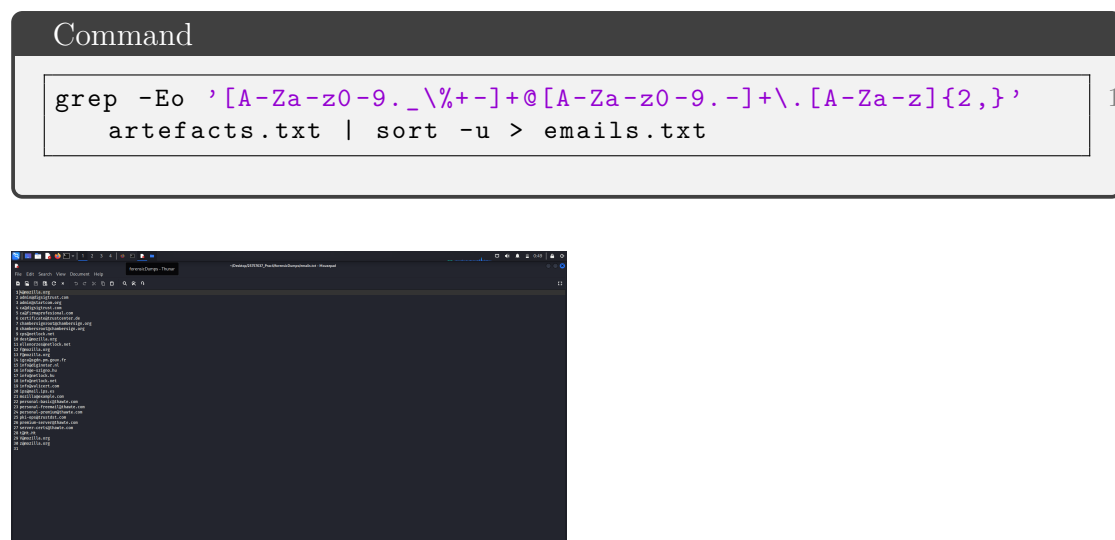


Figure 20: Screenshot- email addresses

The following email addresses were extracted from memory and are associated with known certificate authorities or digital signature providers. These suggest the presence of certificate metadata, PKI validation routines, or cryptographic libraries loaded in memory.:

- admin@digisigtrust.com, ca@firmaprofesional.com,
- certificate@trustcenter.de, info@valicert.com,
- pki-ops@trustdst.com

- personal-basic@thawte.com, server-certs@thawte.com,
info@diginotar.nl, info@netlock.net

The following addresses likely originate from Mozilla certificate store structures, possibly loaded during browser or system trust validation:

- 4@mozilla.org, f@mozilla.org, V@mozilla.org, z@mozilla.org,
dest@mozilla.org, mozilla@example.com

4.3.3 Suspicious or unusual addresses

- t@Ht.Ht - Syntactically valid but non-standard and potentially fabricated.
- ips@mail.ips.es - Domain legitimacy should be verified, as it may be linked to attacker infrastructure.

One notable entry, `info@diginotar.nl`, is historically linked to the DigiNotar compromise of 2011, where fraudulent certificates were issued for major domains. Its presence may indicate legacy certificate data or malicious reuse of compromised trust anchors.

Other addresses, such as `ca@firmaprofesional.com` and `admin@digisigtrust.com`, are tied to legitimate certificate authorities and do not inherently suggest malicious behavior. However, their presence confirms that PKI-related libraries were active in memory.

Unusual entries like `t@Ht.Ht` and `ips@mail.ips.es` warrant further scrutiny due to their non-standard formatting and potential links to attacker infrastructure.

4.3.4 Command Artefacts

```
grep -Ei
'cmd\.exe|powershell|ftp|netstat|whoami|tasklist|ping|curl|wget'
artefacts.txt | sort -u > commands.txt
```

1

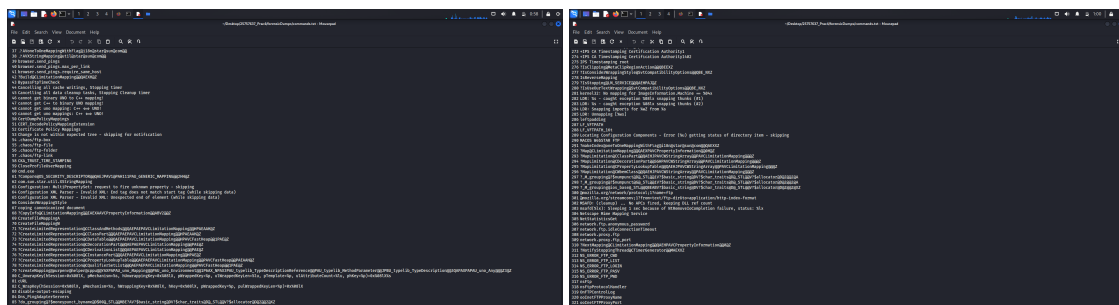


Figure 21: Screenshot- commands output

The following strings indicate attacker-relevant command-line activity. These suggest potential remote control, data staging, or payload retrieval via command-line interfaces:

- `cmd.exe` - Windows command shell
- `ping` - Used to test connectivity or discover internal IPs
- `wget`, `curl` - Tools for downloading payloads or exfiltrating data (variants include `WgetCasePreservingURL`, `_wgetenv`, etc.)

The output includes numerous FTP-related strings:

- `ftp://`, `FtpCommandA`, `FtpOpenFileW`, `FTP error while logging in`
- `network.proxy.ftp`, `xmlNanoFTPConnect`, `nsFtpProtocolHandler`

These artefacts strongly suggest:

- FTP libraries were loaded into memory
- Possibly active FTP sessions or attempted connections
- Use of FTP for file transfer, common in malware staging or exfiltration

The following strings were identified (certificate and timestamping infrastructure):

- `Thawte Timestamping CA`, `Verisign Time Stamping Authority CA`
- `SoftpubLoadSignature`, `PKIX_PL_Cert_GetPolicyMappings`

These indicate:

- Active cryptographic libraries
- Certificate validation routines
- Possible timestamping of signed payloads or documents

The following strings suggest manipulation of Windows internals:

- `cannot get uno mapping: C++ <=> UNO!`
- `LDR: caught exception %08lx snapping thunks`
- `CreateFileMappingA`, `OpenFileMappingW`

These may reflect:

- DLL injection or manipulation of Windows internals
- Memory mapping operations
- Abuse of Windows exception handling mechanisms

4.3.5 URL Artefacts

```
grep -Eo 'http[s]?://[^\s]+|www\.[^\s]+' artefacts.txt | sort -u > urls.txt
```

1

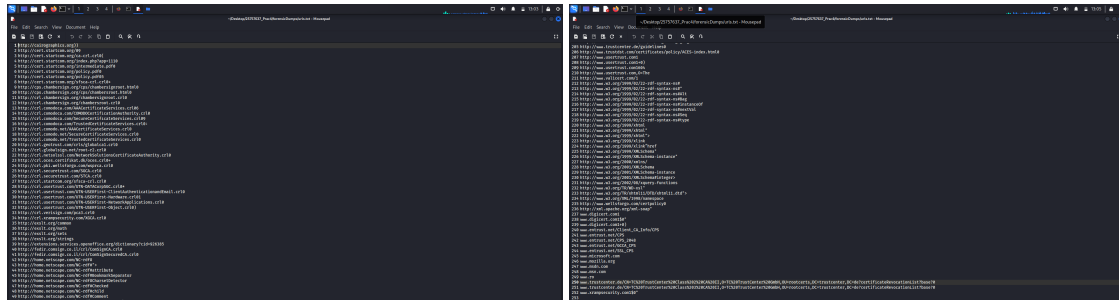


Figure 22: Screenshot- commands output

The memory dumps revealed URLs used for certificate revocation and validation, including the following, and these suggest an active PKI infrastructure and validation of signed payloads or communications.:

- CRL/OCSP endpoints: [http://crl.comodoca.com/...](http://crl.comodoca.com/), [http://crl.verisign.com/...](http://crl.verisign.com/), <https://ocsp.quovadisoffshore.com>, [http://crl.usertrust.com/...](http://crl.usertrust.com/)
- Timestamping: <http://www.ips.es/...Timestamping.crl>

URLs referencing CA policy and trust anchors. Their presence supports certificate handling, either legitimate or spoofed:

- [http://www.trustcenter.de/...](http://www.trustcenter.de/), <http://www.chambersign.org>, <http://www.firmaprofesional.com>, [http://www.certicamara.com/...](http://www.certicamara.com/)

Metadata URLs from OpenOffice and Mozilla, likely loaded by browsers, document processors, or mimicked by malware:

- [http://openoffice.org/...](http://openoffice.org/), [http://www.mozilla.org/...](http://www.mozilla.org/), [http://home.netscape.com/...](http://home.netscape.com/)

Potentially risky or malformed entries:

- <http://wpad/wpad.dat> - Proxy auto-discovery, exploitable in MITM attacks

- `www.ro`, `wwwback.ips.es/...` - Incomplete or unusual domains, worth verifying

The presence of certificate validation endpoints, CA repositories, and application metadata confirms cryptographic and trust-related activity in memory. Suspicious domains such as `wpad.dat` may indicate proxy manipulation or legacy misconfigurations. These findings support further inspection of trust chains and network behavior.

5 Conclusion

The forensic investigation confirms that the Windows XP workstation was compromised. Evidence includes injected code in critical system processes (`'winlogon.exe'`, `'csrss.exe'`), suspicious DLLs loaded into authentication contexts, and external network connections initiated by non-network services such as the Java updater). Artefact analysis revealed attacker-relevant commands (`'curl'`, `'wget'`, `'ftp'`) and legacy certificate infrastructure linked to known breaches. These findings collectively demonstrate unauthorized activity, process tampering, and potential data exfiltration.